Project B - Semester 2, 2026

Table of contents

1 Univariate integer-coefficient polynomials **1**

2 Project Overview **2**
- 2.1 Repository Structure . . . 3
- 2.2 Types . . . 3

3 Tasks (100pts) **3**
- 3.1 **Task 0 (7pts) - Setting up your GitHub repo for hand-in** . . . 4
- 3.2 **Task 1 (12 pts) - Working in an existing codebase** . . . 4
  - 3.2.1 Study the Repo . . . 4
  - 3.2.2 Pretty Printing (8 pts) . . . 4
  - 3.2.3 Showing overflow for `PolynomialDense` (4 pts) . . . 4
- 3.3 **Task 2 (25pts) - Refactoring the data structure used** . . . 5
  - 3.3.1 Sparse Polynomials . . . 5
  - 3.3.2 Constructors (5pts) . . . 5
  - 3.3.3 Operations (10pts) . . . 5
  - 3.3.4 Checking it works (10pts) . . . 5
- 3.4 **Task 3 (25pts) - Optimising Algorithms** . . . 5
  - 3.4.1 Vectorising `PolynomialDense` Addition (5pts) . . . 6
  - 3.4.2 Fast Evaluation (6pts) . . . 6
  - 3.4.3 Specialising `PolynomialSparse` Addition/Multiplication (8pts) . . . 6
  - 3.4.4 Fast Powers (6pts) . . . 7
- 3.5 **Task 4 - Implementing $\mathbb{Z}_p$** (15pts) . . . 7
  - 3.5.1 Constructors (3pts) . . . 8
  - 3.5.2 Miscellaneous functions (4pts) . . . 8
  - 3.5.3 Field Operations (4pts) . . . 8
  - 3.5.4 Checking that it works (4pts) . . . 9
- 3.6 **Task 5 (16 pts) - Using $\mathbb{Z}_p[x]$** . . . 9

1 Univariate integer-coefficient polynomials

(last edit: Sep 24, 2026)

Individual work: This is an individual work assignment. Plagiarism will *not* be accepted. Nevertheless, feel free to consult with friends or classmates via Ed and other means about how to go about certain tasks.

Note: The teaching staff will only answer questions (via Ed, consultation hour, or practicals) regarding this assignment up to late Wednesday evening before the due date.

Submission format: This assignment should be submitted

1. via your GitHub repo and

2. via Blackboard as a PDF file alongside a voice recording (state that the work is your own. Say what was easy, what was hard, what you found useful, and what less. In case you used LLMs (chatGPT etc…), make sure to mention how you used it and where it was useful (or not)).

Specific instructions for the GitHub repo are below. It is important that the GitHub repo be **private** and created within the MATH25042026 github organization. It is also important to name the repo exactly as `PROJECTB-2504-2026-<<FIRST NAME>>-<<LAST NAME>>`. So for example if your name is "Pierre-Simon Laplace", the repo name should be `PROJECTB-2504-2026-Pierre-Simon-Laplace`.

The PDF must have the following components in this order:

1. Your name, student number, and assignment title (Project B - MATH2504 - 2026) on the top.
2. A (clickable) link to your GitHub repo.
3. Screenshots or copy/paste of the output for each `example_script_task_i.jl`, written answers to specific questions and any other outputs requested within this specification (e.g., benchmarks). These should be presented in the order they appear below.
4. A screenshot or copy/paste of the output of `test/runtests.jl` at the end of the PDF (this should be run as `julia test/runtests.jl -v` if Task 5 is complete and `julia test/runtests.jl -vp` otherwise).

Note: *organize your PDF submission into clearly marked sections. Use headings in the format* ***`Task i`*** *or* ***`Task i.j`*** *for each part of the assignment.*

Describe briefly the new snippets of code that you created. It is also a place to answer any questions that require output, or description.

The recommended way to make the PDF file is via a Jupyter notebook where you copy in some code and output into the notebook, and in certain cases use `include()` to run Julia code if appropriate. Also comment on questions in this PDF (e.g. when asked to answer things not via code). If desired keep this jupyter notebook (`.ipynb` file) in the repo. However, this Jupyter notebook will not be checked (only the PDF file), and it is not required to be a "runnable" notebook. In any case, please avoid pixilated screenshots of code (this page may be useful), so for example if you choose to format your PDF file in MS-word (instead of Jupyter), make sure the code is clean, formatted, and readable.

Feedback will be given by the teaching staff by annotating **selected parts** of your PDF file via Blackboard. The teaching staff will also inspect your GitHub repo. A very readable and clean PDF file is important. Note that in printing Jupyter to PDF (or exporting to PDF) there may sometimes be excessive white space. Do not worry about this extra white space; it is not a problem.

Note: *long lines of code in Jupyter notebooks may be truncated by the width of the page when converted to a PDF. Please be aware of this and format your notebook (i.e., split code over multiple lines if needed) as appropriate.*

Weights and Marking Criteria: Total number of points: *100*.

- *7 points* are allocated for setting up GitHub according to the instructions and following the submission instructions.
- *93 points* are allocated for the polynomial project itself. Points are awarded for completing each of the tasks with full marks given to clean readable code.
- In general, points **will be** deducted for sloppy coding style. Make sure your code is properly indented, uses sensible and consistent variable names and write code that is in general, clean and consistent.

---

2 Project Overview

This project requires working in a somewhat larger pre-existing repository and making changes/improvements. The context of the project is *Computer Algebra*, specifically *univariate polynomials* with integer coefficients.

The repository is structured as an actual Julia package. As such, your tasks will involve both understanding how to work with such a repository without violating the existing style and structure, as well as understanding and expanding the underlying computer algebra.

As in a real project, you are not required to understand the underlying mechanics behind every file or function. Understanding the inputs and outputs to functions, and the files relevant to your tasks is sufficient to work in the codebase.

In general, algebraic methods are either only computationally tractable (or even possible) when efficiently representing/computing with polynomials and utilising number theoretic techniques. Your tasks will involve optimising both data structures and algorithms. In addition, several functions have been provided that only work over $\mathbb{Z}_p[x]$ (or a field in general), but the implementation of $\mathbb{Z}_p$ is left to you.

2.1 Repository Structure

The top level tree of the repository looks as follows:

```
.
  LICENSE
  Project.toml
  README.md
  scripts
  src
  test

4 directories, 3 files
```

The `src` (source) directory contains the main package. `test` is as named, a selection of tests which can be run (see `test/README.md` for details) and `scripts` is a place for you to add in small code snippets.

The file `src/PolynomialAlgebra.jl` is the main entry point for the package, and contains the module definition alongside importing all relevant dependencies. The files `_includes.jl` import the files at each folder such that the dependencies are imported in the correct order. If new files/folders are added, these must be updated.

Folders are further subdivided based on the type/functionality they relate to. You should explore these to understand at a high level the purpose of each folder/file.

2.2 Types

The abstract type Polynomial{C} is defined as common super-type for all concrete implementations of polynomials with coefficients of type `C`. The type `Term{C}` is used to represent a term such as cx^n where `c::C` is the coefficient and `n::Int` the degree. A concrete implementation of `Polynomial{C}` is provided in the codebase. It relies on a representation of all – zero and non-zero – terms in the polynomial as `Vector{Term{C}}` and is therefore called `PolynomialDense{C}`.

Refer to Unit 6 for details about polynomial algebra and represenation of polynomials.

3 Tasks (100pts)

Follow the supplied type names below. These are given so that tasks have backwards compatibility. Note that in an actual project, you would sometimes replace older types with newer ones.

Read all the tasks before starting - only some tasks are dependent on previous ones so they do *not* necessarily have to be done in order.

3.1 Task 0 (7pts) - Setting up your GitHub repo for hand-in

- Ideally use the same account you used for BigHW and Project A.
- **Duplicate** the template repository (the codebase) https://github.com/MATH25042026/MATH2505_2026_ProjectB_template as described here and make sure the **new repository is private**.
- Do **not** make any changes (i.e., commits) to the repo after the project due date.
- Create a local clone of the repo. It is *recommended* that you use the `git` command line to make changes/additions to the assignment and submission. However you are free to use any other mechanism (VS-Code, GitHub desktop, etc). You should make intermediate commits to demonstrate your development process.

Your GitHub repo must be formatted like the original repo.

- Ensure that there aren't excessive files in your submission repo. An exception is perhaps a Jupyter `.ipynb` file if you choose to use it for creating the PDF. Use the `.gitignore` file to ensure `git` does not commit additional files and output files to your `repo`.

Note, in any of the following tasks, you may modify which functions are exported in `src/PolynomialAlgebra.jl`. You should only export functions the user may reasonably want to access (i.e., `to_superscript` is used internally, but shouldn't be exported by a polynomial algebra library).

3.2 Task 1 (12 pts) - Working in an existing codebase

This task is for getting the repo to run and improving pretty printing.

3.2.1 Study the Repo

Once you create your own repository, first read the file `README.md`. Then make sure the repository works by running `julia test/runtests.jl -vp` and `julia scripts/example_script.jl`. At this point simply study the functionality of the repository by looking at the function signatures (not implementations) to understand what the repo can do.

3.2.2 Pretty Printing (8 pts)

The current `show`/`print` methods do not produce particularly nice outputs. Instead, we want polynomials to be rendered similarly to the following,

$$-3x^{10} + 3x^5 - x^3 + 2x - 1$$

1. Improve the "pretty printing" of the polynomials by rewriting (refactoring) the `show` methods for both `Term{C}` and `Polynomial{C}`. Use Unicode via the provided function `to_superscript` to display exponents appropriately.
2. Implement subtraction to facilitate easier constructions of polynomials (e.g., `2x - 5` in the original codebase will error although `2x + (-5)` does work).
3. Create your own example script (called `scripts/example_script_task_1.jl`) and include functionality demonstrating the correctness of your "pretty printing" implementation. You should demonstrate this works by constructing (at least) *three* polynomials and including computations of the sum, product and derivative (show as many polynomials as is required to demonstrate all special cases relevant for pretty printing). Following this, verify (in the same script) that `derivative` is working correctly with respect to the product rule.

3.2.3 Showing overflow for `PolynomialDense` (4 pts)

Exact computation requires polynomials with (truly) integer coefficients and thereby we must use `BigInt` rather than (the finitely many integers comprising) `Int`.

1. At the end of `example_script_task_1.jl`, demonstrate using five hard-coded examples (i.e, explicitly write out the polynomials and do not use `rand` to generate them) of `PolynomialDense{Int}` addition and multiplication that overflow by making the coefficients excessively large.
2. Demonstrate (within the same script) that by changing the coefficients to `BigInt`, this overflow does not occur.

Note: remember to include the final output of `example_script_task_1.jl` in your PDF submission.

3.3 Task 2 (25pts) - Refactoring the data structure used

3.3.1 Sparse Polynomials

The current implementation for `PolynomialDense` uses a `Vector` to store the term of degree k at position $k+1$. This is optimal for dense polynomials (those with mostly *nonzero* coefficients) but our (later) factoring algorithms rely heavily on manipulating *sparse polynomials* like $x^n - 1$. In this task we address this deficiency.

Note: you will use the provided type `Heap`, if you wish to modify it check with course staff first.

3.3.2 Constructors (5pts)

1. Within the directory `src/concrete`, create a new directory `sparse` within which sparse polynomials will be implemented. Create all files/subfolders as necessary (see the folder `src/concrete/dense` for what needs to be included). Files/folders should be named appropriately.
2. Create inner and outer constructors for a new type, `PolynomialSparse{C} <: Polynomial{C}`. This type (unlike `PolynomialDense`) should store a `Heap{Term{C}}` rather than a `Vector{Term{C}}`. The max heap (which you can find in `src/utils/heap.jl`) should be used to accomplish this.

Note: Zero terms should be excluded from storage, i.e, for the polynomial x^3+1, the underlying representation for your `PolynomialSparse{C}` should *not* contain a zero term for $0 \cdot x^2$ or $0 \cdot x$.

3.3.3 Operations (10pts)

1. Add in all *necessary functions* (i.e., those for which existing functions defined over the abstract type `Polynomial{C}` are insufficient) in your `polynomial_sparse.jl` definition file to account for the new underlying representation. For example, `iterate` must be overloaded to account for the fact that the `Heap` is not strictly ordered.
2. Add in all required arithmetic functions (at this stage these do *not* need to be fast) and account for your new underlying representation (for example, `+` can no longer use vector indexing). *Note: you will need to compare optimised versions of addition and multiplication of sparse polynomials in the next task, so for now you should only implement the minimum required functionality of polynomial-term addition.*

3.3.4 Checking it works (10pts)

1. Replicate the `example_script_task_1.jl` for `PolynomialSparse` in a file called `example_script_task_2.jl`.
2. Show (using `BenchmarkTools`) that there are examples where `PolynomialSparse{Int}` outperforms `PolynomialDense{Int}` for both time and memory usage. *Do this only for cases where `PolynomialSparse` and `PolynomialDense` do not overflow.*
3. Compare (at least) three pros and/or cons (*in a table within your PDF*) between the dense and sparse representations.

3.4 Task 3 (25pts) - Optimising Algorithms

Many of the algorithms provided within the codebase are not optimised and should be replaced by optimised versions. **For each of the algorithms described below, complete the following tasks** (*note - read*

in full the relevant algorithm description on what is considered sufficient testing/benchmarking and which functions are to be optimised before starting the sub-task):

1. (*PDF Submission*) Benchmark the existing implementation using `BenchmarkTools` and report the results.
2. Refactor all specified functions into the more efficient algorithm.
3. You may use the provided tests to check correctness (run `julia test/runtests.jl -vp`), but these may be inadequate. If stated below, you should add the further specified tests, but it is recommended that you extend the existing tests regardless if you feel they are insufficient (i.e., add new `@testset` blocks). You will only be marked on the correctness of your implementation and any tests that have been explicitly stated below (i.e., we will *not* mark any other additional tests you write regardless of whether they are correct). You may choose which `Polynomial` concrete subtype to run your tests on if the algorithm is implemented at the abstract level. For any new tests, create a file `test/algorithms_test.jl` and include your tests there (these should be runnable via `julia test/runtests.jl -vp`).
4. (*PDF Submission*) Benchmark your new implementation using `BenchmarkTools` and report the results.
5. (*PDF Submission*) State the time complexity of the initial implementation and your new optimised implementation.

Note: the following algorithms are intended to be listed in the order of easiest to hardest, but do not depend on each other and hence you may do/skip them in any order.

3.4.1 Vectorising `PolynomialDense` Addition (5pts)

Our current implementation for adding two `PolynomialDense{C}` together simply delegates the task to the abstract implementation. The abstract implementation however, uses significant amounts of memory and time by making copies of polynomials at each stage, and does not leverage Julia's native speed with vectors.

Implement a version of addition between two `PolynomialDense{C}` (they *must* be parameterised with the same type parameter) that is vectorised and in particular, does not require explicit iteration (i.e., you shouldn't need a `for` loop) through the underlying vectors of either polynomial.

For benchmarking, find a single instance where the existing implementation takes on average at least 1s to run. Demonstrate that your implemention is indeed an optimisation, and report the memory usage before and after your optimisation.

3.4.2 Fast Evaluation (6pts)

Our current evaluation strategy for a polynomial $a_0+a_1\cdot x+...+a_n\cdot x^n$ is for the i'th term, take x to the power i, then multiply by the i'th coefficient a_i. This however, is inefficient - Horner's method offers a much faster evaluation method, by implicitly utilising the following representation: $(...(a_n\cdot x+a_{n-1})\cdot x+...+a_1)\cdot x+a_0$.

Utilise this algorithm to optimise the existing implementation of `evaluate`. This algorithm should be implemented at the abstract level, but should improve the runtime for both sparse/dense representations simultaneously.

For benchmarking, find a single instance where the existing implementation takes on average at least 1s to run. Demonstrate that your implementation is indeed an optimisation.

Test your implementation of `evaluate` for at least 10^2 random polynomials (your tests should not take more than 30s to run on your computer).

3.4.3 Specialising `PolynomialSparse` Addition/Multiplication (8pts)

Note: to implement either of these algorithms well, you should study the provided functions in `src/utils/heap.jl` for manipulating the underlying data of the heap efficiently.

Addition `+(PolynomialSparse{C}, PolynomialSparse{C})`

Our current implementation for adding two `PolynomialSparse{C}` together once again delegates the task to the abstract implementation. As before, the abstract implementation is inefficient.

The time complexity of a single `push`/`pop` operation on a `Heap` is $O(\log_2 n)$ where n is the number of terms in the heap (`peek` is $O(1)$). We can use this fact to produce an addition algorithm between two sparse polynomials by repeatedly popping from each heap, and adding the two terms together if they have the same degree. You should attempt to minimise the additional memory used (i.e., as few or no `deepcopy`/`copy`'s if possible).

For benchmarking, find a single instance where the existing implementation takes on average at least 1s to run. Demonstrate that your implemention is indeed an optimisation.

Multiplication `*(PolynomialSparse{C}, Term{C})`

As above, multiplication between a term and sparse polynomial is currently delegated to the abstract methods. This can be significantly optimised by understanding that *pointwise multiplication between a term and vector of terms, will not change the relative ordering of the vector.*

I.e., given the terms $3x^2$, $2x^5$ and x^6, we have that $x^6 > 2x^5$ (where we order terms by degree). We also know that $(3x^2) \cdot x^6 > (3x^2) \cdot (2x^5)$. Leverage this property to produce a vectorised version of term-polynomial multiplication (the provided `Heap` data structure provides you with methods to achieve this).

For benchmarking use multiplication between two sparse polynomials (i.e., not between a term and a polynomial), find a single instance where the existing implementation takes on average at least 1s to run. Demonstrate that your implemention is indeed an optimisation.

3.4.4 Fast Powers (6pts)

As discussed in class, if you have a polynomial f and raise it to the power 2^n then instead of doing about 2^n multiplications you can do about n using **repeated squaring**. E.g., to compute f^8, we compute $f^2 = f \cdot f$, then $f^4 = f^2 \cdot f^2$ and finally $f^8 = f^4 \cdot f^4$. This idea can be extended to computing f^n for any $n \in \mathbb{N}$! Utilise this algorithm to optimise the existing implementation of `^`. You may wish to short-circuit computations of f^0 and f^1 (this is not required). This algorithm should be implemented at the abstract level (i.e., `Polynomial{C}`).

For benchmarking, find a single instance where the existing implementation takes on average at least 1s to run. Demonstrate that your implemention is indeed an optimisation.

Add new tests for your implementation of `^` using at least 10^2 generated polynomials (your tests should not take more than 30s to run on your computer).

Note: To construct polynomials where you can easily check the correctness of the result, you may want to use the Binomial Theorem alongside polynomials $(1 + x)^n$.

3.5 Task 4 - Implementing $\mathbb{Z}_p$ (15pts)

We have provided an implementation of the *Cantor-Zassenhaus factoring algorithm* which allows for factoring polynomials when the coefficient ring is the field $\mathbb{Z}_p$. Previously, this course delved into the algorithm itself (and the corresponding algorithm to reconstruct a full factorisation over the integers). However, this was too involved (feel free to try to understand the algorithm/implementation if you wish to do so - but this is not necessary and will not be assessed) and instead we will simply utilise the implementation as a *blackbox*.

Until now, your polynomial implementation has assumed integer coefficients and (fatally) $\mathbb{Z}[x]$ is *not* a Euclidean domain. This means we do not yet have access to `gcd` (or even `div`/`rem`) on polynomials which is a critical component of the factoring algorithm. If we take our coefficients from a *field* like $\mathbb{Z}_p$, for p a prime, then the resulting $\mathbb{Z}_p[x]$ *is* a Euclidean domain. We therefore need to implement some way to work with $\mathbb{Z}_p[x]$.

Your task is to implement the coefficient field $\mathbb{Z}_p$, such that in the next task we can utilise it to factor polynomials.

Note: you should aim to implement all functions in this section as efficiently as you can. Additionally, if you can implement any of the following functions in an equivalent (in particular, multiple dispatch should work with the same effectiveness) manner but a simpler type signature, you may do so.

To begin, view the provided stub and initial implementation in `src/coefficients/z_mod_p/z_mod_p.jl`.

Note: that `N` is a value type and thus it must be specified as a constant, it cannot be generated (e.g. via rand) at runtime.

3.5.1 Constructors (3pts)

An *inner* constructor has been provided for you. Create *outer* constructors that may be used to easily construct `0 (mod p)` and `1 (mod p)`. The following code should work at a minimum:

```
z5 = ZModP{5}
a = z5(3)
b = z5(4)

# Should be 0 (mod 5)
zero(z5)
zero(a)

# Should be 1 (mod 5)
one(z5)
one(a)
```

You may at this stage add any other reasonable outer constructors for convenience (*not assessed*).

Notes:

In all of the following, it is up to you to determine exactly which function type signatures are required - remember that you can always return and add more if necessary later on. Remember also to prefix relevant functions with `Base.`, e.g., `Base.show` or `Base.:(+)`.*

It is possible to complete this task only implementing the explicitly mentioned operations between objects of type `ZModP{N}`.

Your pretty printer may stop working optimally during these tasks for objects of type `Polynomial{ZModP{N}}` - you will not be marked down for this provided the outputs are still correct and no errors occur (e.g., $x + 6 \pmod 7$ may be rendered as $x + -1$).

3.5.2 Miscellaneous functions (4pts)

Define methods for `show` and `==`.

3.5.3 Field Operations (4pts)

Implement basic arithmetic for $\mathbb{Z}_p$ including `+`, `-`, `*`, `inv`, `÷`, `/` and `^` .

Note: you should optimise `^` similarly to the fast power section of Task 3.

Hint: Recall division in a field is simply multiplication by the inverse. See `int_inverse_mod` in `src/utils/general_alg.jl`.

All the above operations should act as they do in $\mathbb{Z}_p$. E.g., $3 \cdot 2 = 6 \equiv 1 \, (mod 5)$ should correspond to the code:

```
a = ZModP{5}(3)
@show a * 2 # should print 1
```

3.5.4 Checking that it works (4pts)

1. Create `scripts/example_script_task_4.jl` and in this file, demonstrate *all* basic modular arithmetic operations working correctly (each at least once).
2. Create `test/z_mod_p_test.jl` containing tests which run *all* operations on `ZModP{N}`. Your tests must run on at least 10^3 randomly generated integers in the range $[-10^4, 10^4]$ inclusive and use at least 4 different primes. Utilise the in-built `mod` function on integers to verify correctness of your operations.
3. Ensure your implementations pass these tests, and that they run with the command `julia test/runtests.jl -vp`.

Note: make sure to add the output of `example_script_task_4.jl` to your PDF submission.

3.6 Task 5 (16 pts) - Using $\mathbb{Z}_p[x]$

Before starting this section, ensure all tests pass when run with command `julia test/runtests.jl -vp`.

Now that we have `ZModP` as a subtype of `Number`, we can use this as the coefficient type of our polynomials. E.g., `PolynomialSparse{ZModP{Int, 5}}`. *Try experimenting in the REPL with this before attempting the rest of this section.*

Implement the following function for polynomial division:

```
function Base.divrem(f::P, g::P)::Tuple{P, P} where {P <: Polynomial}
```

It returns the tuple of div(isor) and rem(ainder). Make sure the tests pass when run with command `julia test/runtests.jl -v`.

Within a file `scripts/example_script_task_5.jl`, demonstrate the division algorithm over a field $\mathbb{Z}_p[x]$ for some $123 < p < 423$.